

Ghost — Modular Build Plan

Derived from: Master Platform Architecture spec (docx), Full UI/UX Blueprint (index.html, 409 pages / 61 architecture concepts), Complete Full-Scrolling Wireframe (index2.html, 99 source-mapped contract blocks), Developer UI/UX Spec (890 files, 66 contract-mapped screens).

Every module below is independently buildable, independently testable, and has an explicit done-test. Dependencies are named. Nothing depends on a module below it in the same tier.

PART 0 — BLOCKING CONFLICTS (resolve before writing code)

These are real contradictions between your own documents. Each one will cause a rewrite at month 4 if not settled now.

C1. Two incompatible schemas for the same concepts

Concept	Master spec (docx §8)	UI atlas (index2 / dev spec)
Business identity	entities , entity_types , entity_relationships	businesses , locations
Membership	memberships , roles	workspace_memberships , roles
Permission	permissions , grants , role_bindings	permission_catalog , permission_grants , permission_requests
Action run	actions , action_runs , approvals	executions , execution_steps , execution_targets , approval_requests
Proof	receipts , receipt_evidence	ledger_events , evidence , verification_results
Provenance	sources , provenance , verification_records	source_observations , canonical_fields , canonical_authorities , canonical_conflicts

Pick one. My recommendation: **the atlas names win on the operational side** (`executions` , `ledger_events` , `verification_results` , `source_observations`) because 142 tables and 124 endpoints are already specced against them. **The docx wins on the identity side** (`entities` as universal registry) because a generic entity registry is what lets business identity, customer identity, device identity, and GCR consumer identity share one graph. Then `businesses` becomes a typed projection over `entities` , not a parallel table.

C2. Is `entities` universal, or is `businesses` the root?

The docx describes one universal entity registry (person, business, device, product, location, app). The atlas writes directly to `businesses` . These produce completely different systems. If GCR consumer identity, the personal Ghost (Ghost Gate, §59, page #109), and business identity are meant to live in the same graph, you need the universal registry. If not, you have two products.

C3. The vocabulary is not frozen

Master spec Phase 1 is “freeze vocabulary.” The developer spec reports **66 of 409 screens contract-mapped — 343 unmapped**. The atlas reports 99 contract blocks. So somewhere between 66 and 99 of 409 surfaces have a defined backend contract. Phase 1 is the actual blocking task and it is ~20% done.

C4. Your execution risk is inverted

Across the 99 source-mapped contract blocks, **179 of 189 capability bindings execute against `database`** . Only 3 are `execution-control` ; 7 are `none` . Zero are browser or Android in the mapped set.

Read that again. The specified product is overwhelmingly canonical data + permissions + CRUD. Browser and Android automation — the part carrying platform-ToS risk, AppMap maintenance cost, and fleet support burden — is a small appendix to the actual surface area you designed. Build accordingly. Executors come late, not early.

PART 1 — INVENTORY (what “all of it” actually is)

Artifact	Count	Source
Canonical tables named	142 distinct	screen specs + atlas contracts

API endpoints specced	124 distinct	/v1/... across screen specs
Realtime events named	56 distinct	screen spec contract blocks
Capabilities named	43 in atlas contracts + 15 in spec Appendix A	~50 after dedupe
Permission scopes	16	screen spec read-permissions
Reusable components	10	atlas #370–380
Surface block types	15	docx §14.1
UI page concepts	409	atlas / manifest
Architecture concepts	61	manifest
Backend modules	18	atlas #99–#147
Implementation phases	20	docx §68

Page-count deflation. Of the 409: 50 industry + 39 vendor + 17 connected-system = **106 pages that are one template plus a content row each** (verified: identical Actions/States fields across all). Plus ~50 marketing pages (a website, not product), 10 demo businesses (one template + seed data), 72 "Platform / Reference" explainers. **Distinct product surfaces: roughly 60–70 screens.**

PART 2 — THE MODULES

TIER 0 — Frozen contracts (specification, not code)

Nothing below this tier can start until these are versioned and reviewed. This is Phase 1 of the docx and it is the whole ballgame.

M0 · Vocabulary & ID scheme Owns: the 20 contract nouns (Entity, Resource, Capability, Action, Event, State, Permission/Grant, App, Surface, Executor, Endpoint, Workflow, Policy, Receipt, Provenance, Schema). `ghost://` URI format for every entity type. Depends on: C1,

C2 resolved. Done when: a written spec doc where no term has two meanings, and every one of the 142 tables maps to exactly one noun.

M1 • Schema registry Owns: `core.business@1`, `core.customer@1`, `core.location@1`, `core.product@1`, `core.availability@1`, plus industry schemas (`restaurant.menu@2`, `charter.trip@1`, `music.song_request@1`, `lodging.*`). Versioning and migration rules. Done when: two independently written apps reading `core.customer@1` produce identical field interpretation.

M2 • Capability catalog Owns: ~50 capability names, typed inputs/outputs, permission requirement, risk class (Low / Routine write / Sensitive / Financial-irreversible / Policy change), idempotency rule, and **verification contract** — how each one proves it worked. Done when: every capability has a named verification method. A capability without one is not defined.

M3 • Event catalog Owns: the 56 event names, the event envelope (`event_id`, `type`, `schema_version`, `tenant_id`, `actor`, `entity`, `occurred_at`, `source`, `payload`, `provenance`). Done when: envelope is frozen and every event has a declared producer.

M4 • Permission catalog Owns: the 16 scopes (`workspace.read`, `onboarding.manage`, `executions.read`, `builder.manage`, `business.profile.read`, `packages.read`, `connections.read`, `automations.read`, `customers.read`, `analytics.read`, `team.read`, `settings.read`, `developer.read`, `agency.clients.read`, `public`, plus writes). Done when: every one of the 124 endpoints declares required scope.

TIER 1 — Foundation (blocks everything)

M5 • Core Postgres + tenancy Tables: `organizations`, `users`, `identities`, `entities`, `entity_types`, `entity_relationships`, `businesses`, `locations`, `addresses`, `phones`, `websites`, `workspace_memberships`, `roles`, `invitations`. Done when: RLS tenant-isolation tests pass; no cross-tenant read is possible with a valid token for another tenant.

M6 • Provenance engine ← *your actual differentiator* Tables: `source_observations`, `canonical_fields`, `canonical_authorities`, `canonical_conflicts`, `canonical_draft_facts`, `canonical_entity_types`, `source_import_batches`. Implements the 9-tier authority order: owner-authenticated → direct operational system → authorized live integration → verified employee → transaction evidence → structured public source → business website → third-party → AI inference. Every material fact carries `source_type`, `source_ref`, `observed_at`, `verified`, `confidence`, `version`. Done when: two sources disagree on Friday hours, the conflict surfaces explicitly, the owner decides, and AI inference can never silently overwrite an owner-verified fact.

M7 · AuthN + sessions Tables: `auth_sessions`, `verification_methods`, `verification_challenges`, `business_claims`. Endpoints: `POST /v1/auth/{email,phone,google}/start`, `POST /v1/session/workspace`, `POST /v1/entities/{id}/claim`. Done when: ownership verification produces a durable authority record, not just a session.

M8 · Permission grants + policy gate Tables: `permission_catalog`, `permission_grants`, `permission_requests`, `permission_requirements`. Start with an internal deterministic policy function returning `AUTO / ASK / NEVER`. Keep the `PolicyProvider` interface so OPA drops in later. Non-negotiable from docx Appendix F: **AI is never the final authorization authority**. Policy accepts structured facts and cryptographic identity only — never natural-language persuasion. Done when: no privileged capability can execute without passing the gate, verified by test.

M9 · Event bus + transactional outbox Tables: `events`, `event_outbox`. Start with Postgres outbox + `LISTEN/NOTIFY`. Keep the interface so `NATS/JetStream` drops in later. Done when: record change and event insert commit in the same transaction; killing the dispatcher mid-run loses zero events.

TIER 2 — Execution spine

M10 · Capability registry + router Tables: `capability_registry`, `capability_implementations`, `executor_routes`. Done when: one capability request resolves to a provider without the caller knowing which.

M11 · Action/execution runtime Tables: `executions`, `execution_steps`, `execution_targets`, `scheduled_jobs`. Postgres job table first. Keep `WorkflowEngine` interface for Temporal later. Done when: idempotency keys prevent duplicate consequential actions; retries know resume-vs-repeat.

M12 · Verification service Tables: `verification_results`. Expected-state contracts, verification readers, expected-vs-observed comparison, `pass / fail / uncertain`. Done when: “the Save button returned 200” and “the destination state now matches” are distinguishable outcomes in the data model.

M13 · Ledger / receipts / evidence Tables: `ledger_events`, `evidence`. Append-only. Records actor, requested action, permission decision, policy version, executor, timing, result, verification, evidence ref. Done when: every consequential action has a receipt and the ledger cannot be rewritten.

M14 · Approval gateway Tables: `approval_requests`, `attention_items`. Owner approves by SMS reply, not by logging into a dashboard. Done when: an `ASK`-class action blocks,

notifies, and resumes from durable state on approval.

TIER 3 — Interfaces

M15 · Control plane API — the 124 endpoints, single entry for dashboard/MCP/SMS/voice/apps.

M16 · MCP gateway ← *missing from your 18 backend modules entirely* One central gateway, tenant-authorized, logical route `/mcp/business/{business_id}` over shared implementation. Resources: menu, hours, availability, events. Tools: `availability.search`, `special.update`, `reservation.create`. This is your stated core thesis — the business defining its own MCP surface — and it has zero contract in the corpus. It should be a Tier 3 module, not a Phase 17 afterthought.

M17 · Realtime channel — the 56 events delivered to clients.

TIER 4 — Surfaces

M18 · Reusable component library (atlas #370–380 — these are your standard library) Approval Gate · Availability Card · Business Hours · Customer Match · Permissioned Message Send · Public Profile Block · Retry + Idempotency · Review Request Step · Source Reconciliation · Verification Step. Every one of the 16 use cases and 11 automations composes from these ten. Build them once.

M19 · Surface renderer — 15 block types: card, accordion, form, table, calendar, gallery, list, chart, map, modal, drawer, button, search, status/badge, media player.

M20 · Owner dashboard shell — Home / Business / Apps / Connections / Automations / Ghost / Devices. Linktree-style expandable modules; tap to expand inline, tap to collapse, open full workspace when needed.

M21 · Public surface + visibility + QR Tables: `public_pages`, `public_page_sections`, `public_page_bindings`, `public_releases`, `visibility_rules`, `qr_codes`, `qr_destinations`. Done when: the website is provably a renderer over canonical data, not a second database.

TIER 5 — Connectivity

M22 · Connection / provider framework Tables: `connections`, `connection_methods`, `connection_health`, `connection_requests`, `connection_drafts`, `provider_catalog`,

`provider_packages , external_accounts , external_entities .`

M23 · Source adapters — each individually modular: iCal, email parser, GBP, Square, Toast, FareHarbor, Peek. One adapter is one module. Start with two.

M24 · Secrets — `secret_refs , credential_bindings .` Envelope encryption in Postgres first; `SecretStore` interface for OpenBao later. Apps receive opaque references, never plaintext.

TIER 6 — Packages

M25 · Package format + installer Tables: `package_catalog , package_versions , package_installs , package_permissions , package_requirements , package_compatibility , package_releases , package_validation_results .` Installation is a permission transaction, not a download.

M26 · App sandbox — rootless container first; WASM/WASI later. Apps get no raw DB credentials, default-deny network, per-app quotas.

TIER 7 — Automations

M27 · Automation engine Tables: `automation_definitions , automation_steps , automation_drafts , automation_installs , automation_runs , automation_schedules , automation_health , automation_templates .` Native WHEN → IF → THEN → VERIFY engine.

M28 · The 11 named automations — Daily Business Brief, Estimate Follow-Up, Event Publisher, Holiday Hours Everywhere, Last-Minute Opening, Maintenance Block, Menu Price Publisher, Post-Trip Customer Flow, Verified Review Follow-Up, Weather Cancellation, Visual Builder. Each is one module composed from M18 components.

TIER 8 — Executors (late, and in this order)

M29 · API/HTTP executor — preferred path always. **M30 · Browser executor** (Playwright) — `BrowserExecutor` interface. **M31 · Device endpoint** — Linux/Pi agent: identity, capability advertisement, commands, health. Powers signage. **M32 · Android executor** — LAST. Carries platform-ToS and account-ban risk to your customers' actual business pages. Not the hero demo. **M33 · AppMaps + compatibility + repair** — `appmaps , compatibility_tests , compatibility_health .` Only meaningful once you have real breakage at fleet scale.

TIER 9 — Deferred (docx §69 says don't build these first; agreed)

M34 Builder · M35 Developer platform · M36 Marketplace · M37 Metering/billing · M38 Node runtime / local box / hardware · M39 Constitution anchoring.

Explicit “not first” list from your own spec: custom motherboard, Ghost Key hardware, hologram accessory, huge marketplace, 50 industry packs, custom blockchain, custom AI model, custom database engine, custom container engine, custom VPN protocol, new browser automation framework.

PART 3 — BUILD ORDER

Sequential (nothing parallelizes around these): M0 → M1 → M2 → M3 → M4 → M5 → M6 → M8 → M9

Then three parallel tracks:

- Track A (spine): M10 → M11 → M12 → M13 → M14
- Track B (interface): M15 → M17 → M16
- Track C (surface): M18 → M19 → M20 → M21

Converge on the proof test, then: M22 → M23 → M25 → M27 → M28 → M29 → M30.

PART 4 — THE PROOF TEST (docx §73)

Install three apps from three independent developers. They must share business identity, permission model, event system, and dashboard with zero pairwise integration code. Then:

1. Replace the reservation app — other apps and data keep working.
2. Replace the website/surface — structured data unchanged.
3. Replace the AI model — capabilities and workflows unchanged.
4. Move the runtime cloud → mini-PC — identity, grants, apps, schemas, workflows, data survive.
5. Remove one app — it loses capability access without taking owner data with it.

If replacing a component requires rebuilding the owner's business, that component is still too tightly coupled.

PART 5 — NON-NEGOTIABLES (docx Appendix F — hold these)

- The app must not own canonical business data.
- The surface must not be the canonical store.
- AI must never be the final authorization authority.
- Third-party apps never receive broad raw database credentials.
- Every cross-app shared concept uses a versioned schema contract.
- Every important action defines verification and idempotency behavior.
- Local/cloud/hardware choice must not change logical contracts.
- An outside developer must be able to build a compatible capability without permission from Ghost Inc. for the protocol itself.